

Reviewer Pack — Shifted Partial of Clause Cubic Lower-Bound Tree-Resolution...

Entry 3cb40990c2eb · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-28 16:59:21 UTC

Shifted Partial of Clause Cubic Lower-Bound Tree-Resolution Leaves

Entry ID: 3cb40990c2eb **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-28 16:59:21 UTC

1. Conjecture

Field A (mathematical branch): Shifted partial-derivatives method in geometric complexity theory (Kayal-Saha-Tavenas 2016; Gupta-Kamath-Kayal-Saptharishi 2014): the dimension $\text{SPD}_{\{\square, k\}}(p) = \dim_Q \text{span}\{x^{\beta \cdot \partial} \alpha \mid |\beta| = \square, |\alpha| = k\}$ of the multiplicative-shift of partial derivatives, a coordinate-free arithmetic-circuit lower-bound functional computable in $O(n^6)$ by rank of an explicit Sylvester-like lift matrix. It strictly extends Mignon-Ressayre Hessian rank ($= \text{SPD}_{\{0,1\}}$ up to constants) and is the dominant algebraic technique used to prove super-polynomial homogeneous depth-4 bounds in the Mulmuley-Sohoni GCT program; rarely deployed in proof complexity.

Field B (complexity object): Tree-like resolution refutation leaf count $L_T(F \vdash \perp)$ of unsatisfiable random 3-CNF F at clause density $\alpha \in [4.5, 5.5]$, equivalently lex-DPLL search-tree leaf count under unit propagation — the canonical proof-complexity proxy for refutation hardness.

Statement:

For an unsat 3-CNF F on $n \leq 16$ vars, form the clause cubic $Q_F(x_0, \dots, x_n) = \sum \{C = (a, b, c) \in F\} \square_a \cdot \square_b \cdot \square_c$ where $\square_i^+ = x_i$, $\square_i^- = x_0 - x_i$, viewed in the monomial basis of $Q[x_0, \dots, x_n]^3$. Define $\psi(F) := \text{rank } M_{\{1,1\}}(Q_F) - \text{rank } \text{Cat}_1(Q_F)$, where $M_{\{1,1\}}$ is the $(n+1)^2 \times (n+3 \text{ choose } 3)$ coefficient matrix whose rows are the cubic coefficients of $x_i \cdot \partial_j Q_F$, and Cat_1 is the $(n+1) \times (n+2 \text{ choose } 2)$ first-catalecticant. Conjecture: $\psi(F) \geq \lceil \log_2 L_T(F \vdash \perp) \rceil$ for every unsat 3-CNF F at clause density $\alpha \in [4.5, 5.5]$; a single instance with $\psi(F) < \lceil \log_2 L_T \rceil$ falsifies it.

Rationale (proposer's reasoning):

Q_F algebraicizes the unsat witness as a degree-3 form; multiplying first partials by linear shifts probes the 'depth-4 expansion' that Gupta-Kamath-Kayal-Saptharishi exploited for the $2^{\sqrt{n}}$ IMM bound, exactly the GCT-aligned nonlinear information missed by the Hessian alone. Subtracting Cat_1 isolates the genuinely shifted contribution and steers clear of the prior Mignon-Ressayre attempt on the blacklist. Tree-resolution leaf count is a sharp proxy for DPLL hardness, so a quantitative ψ -vs- $\log L_T$ link would expose unsat-instance hardness as a Sylvester-rank obstruction.

Taxonomy category: GCT (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): ea8e53168911d3e2

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across $n \in \{8, 10, 12, 14, 16\}$ and $\alpha \in \{4.5, 5.0, 5.5\}$, generate 200 random 3-CNFs per cell; restrict to unsat instances with L_T from memoized lex-DPLL. Compute $\psi(F) = \text{rank } M_{\{1,1\}} - \text{rank } \text{Cat}_1$ (tol $1e-8$). Support requires $\psi(F) \geq \lfloor \log_2 L_T \rfloor$ in $\geq 95\%$ of unsat trials per cell; any seed with $\psi(F) < \lfloor \log_2 L_T \rfloor - 1$ falsifies.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL PROOFS	SAFE	0.90	SAFE	SAFE
ALGEBRIZATION	UNCERTAIN	0.95	SAFE	HITS
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - shifted partial derivatives proof complexity resolution lower bound - catalecticant cubic form 3-CNF refutation tree resolution - arithmetic circuit lower bound technique DPLL random 3-SAT clause density

Top relevant hits considered: - [<http://arxiv.org/abs/1606.05050v1>] Proof Complexity Lower Bounds from Algebraic Circuit Complexity - [<http://arxiv.org/abs/cs/9906008v2>] A Lower Bound on the Average-Case Complexity of Shellsort - [<http://arxiv.org/abs/1801.08709v2>] Adaptive Lower Bound for Testing Monotonicity on the Line - [<http://arxiv.org/abs/2404.04038v1>] Refutability as Recursive as Provability - [<http://arxiv.org/abs/0804.4584v1>] Feature Unification in TAG Derivation Trees - [<http://arxiv.org/abs/1608.07792v1>] A Generalized Resolution Proof Schema and the Pigeonhole Principle - [<http://arxiv.org/abs/1710.07476v6>] Planar 3-SAT with a Clause/Variable Cycle - [<http://arxiv.org/abs/1908.05361v2>] Placing quantified variants of 3-SAT and Not-All-Equal 3-SAT in the polynomial hierarchy

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=124, elapsed=180.1s

5.1 Generated Python source

```
import random
import math
from collections import defaultdict

def gaussian_elimination(A):
    n = len(A)
    m = len(A[0])
    rank = 0
```

```

for i in range(n):
    max_row = i
    for j in range(i+1, n):
        if abs(A[j][i]) > abs(A[max_row][i]):
            max_row = j
    A[i], A[max_row] = A[max_row], A[i]
    if A[i][i] == 0:
        continue
    rank += 1
    for j in range(i+1, n):
        factor = A[j][i] / A[i][i]
        for k in range(m):
            A[j][k] -= factor * A[i][k]
return rank

def matrix_multiply(A, B):
    n = len(A)
    m = len(B[0])
    p = len(B)
    C = [[0 for _ in range(m)] for _ in range(n)]
    for i in range(n):
        for j in range(m):
            for k in range(p):
                C[i][j] += A[i][k] * B[k][j]
    return C

def determinant(A):
    n = len(A)
    if n == 1:
        return A[0][0]
    det = 0
    sign = 1
    for j in range(n):
        submatrix = [row[:j] + row[j+1:] for row in A[1:]]
        det += sign * A[0][j] * determinant(submatrix)
        sign *= -1
    return det

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_3cnf(n, alpha):
        m = int(alpha * n * (n - 1) // 2)
        clauses = set()
        while len(clauses) < m:
            a, b, c = random.sample(range(n), 3)
            clause = tuple(sorted([a, b, c]))
            if clause not in clauses:
                clauses.add(clause)
        return clauses

    def lex_dpll(F):
        n = len(F)
        unit_propagation = [False] * n
        for _ in range(2 ** n):
            assignment = [i % 2 == 1 for i in range(n)]

```

```

        if all(any(lit in clause for lit in assignment) or any(-lit in clause for lit in assignment))
            return True, assignment
    return False, None

def build_Q_F(F, n):
    monomials = defaultdict(int)
    for a, b, c in F:
        monomial = (a, b, c)
        monomials[monomial] += 1
    return monomials

def build_M_11(Q_F, n):
    m_11 = []
    for i in range(n + 1):
        row = [0] * ((n + 3) * (n + 2) // 6)
        for a, b, c in Q_F:
            if a == i or b == i or c == i:
                index = (a + b + c - 3) * (a + b + c - 4) // 2 + a + b + c - 1
                row[index] += Q_F[(a, b, c)]
        m_11.append(row)
    return m_11

def build_Cat_1(Q_F, n):
    cat_1 = []
    for i in range(n + 1):
        row = [0] * ((n + 2) * (n + 1) // 2)
        for a, b, c in Q_F:
            if a == i or b == i or c == i:
                index = (a + b - 2) * (a + b - 1) // 2 + a
                row[index] += Q_F[(a, b, c)]
        cat_1.append(row)
    return cat_1

def log2_floor(x):
    if x <= 0:
        return -math.inf
    return math.floor(math.log2(x))

n = random.choice([8, 10, 12, 14, 16])
alpha = random.uniform(4.5, 5.5)
F = generate_3cnf(n, alpha)
unsat, _ = lex_dp11(F)
if not unsat:
    return {
        "metric_name": " $\psi(F)$ ",
        "metric_value": None,
        "instances_tested": 0,
        "conjecture_holds": False,
        "counterexample": "unsat_instance"
    }

Q_F = build_Q_F(F, n)
M_11 = build_M_11(Q_F, n)
Cat_1 = build_Cat_1(Q_F, n)

rank_M_11 = gaussian_elimination(M_11)

```

```

rank_Cat_1 = gaussian_elimination(Cat_1)

psi_F = rank_M_11 - rank_Cat_1
L_T = 2 ** (len(F) / n)
log2_L_T = log2_floor(L_T)

return {
    "metric_name": " $\psi(F)$ ",
    "metric_value": psi_F,
    "instances_tested": 1,
    "conjecture_holds": psi_F >= log2_L_T,
    "counterexample": "" if psi_F >= log2_L_T else f"psi_F={psi_F}, log2(L_T)={log2_L_T}"
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [11, 23, 37, 53, 71]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    total_metric_value = sum(res["metric_value"] for res in results if res["metric_value"] is not None)
    num_trials = len(results)
    mean_metric_value = total_metric_value / num_trials
    std_metric_value = math.sqrt(sum((res["metric_value"] - mean_metric_value) ** 2 for res in results) / num_trials)

    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / num_trials

    if all(res["conjecture_holds"] for res in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif any(not res["conjecture_holds"] and res["metric_value"] < math.floor(math.log2(2 ** (len(F) / n))) for res in results):
        print(f"RESULT: FALSIFIED counterexample=\"psi_F<floor(log2(L_T))-1\" first_failing_seed={seeds[0]}")
    else:
        print(f"RESULT: INCONCLUSIVE insufficient_data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 180s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out after 180s (returncode=124) without producing any RESULT line or data, so the pre-registered support condition cannot be evaluated. No counterexample was observed either, so falsification is not established. | next: Reduce the workload (e.g., shrink to $n \in \{8, 10, 12\}$ and 50 trials/cell, or precompute L_T with a faster DPLL) and raise the timeout so the rank computations and lex-DPLL counts can complete.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	202242
2	preregistration	claude_max	opus	0	0	6579
3	novelty	claude_max	opus	0	0	4251
4	novelty	claude_max	opus	0	0	9554
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16735
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18350
7	judge	claude_max	opus	0	0	5236

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 262947 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/3cb40990c2eb.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/3cb40990c2eb.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/3cb40990c2eb.tar.gz` (if generated)